

1 Algorithms and Cryptography Utilities

1.1 RSA Key Generation: `generateRSAKeyPair()`

1.1.1 Purpose

The RSA key generation algorithm is responsible for creating a public-private key pair for each user. The public key can be distributed openly, while the private key must remain secret.

1.1.2 Mathematical Foundation

Two distinct prime numbers p and q are selected.

The RSA modulus is computed as

$$n = pq. \quad (1)$$

The Euler totient function is then calculated:

$$\phi(n) = (p - 1)(q - 1). \quad (2)$$

A public exponent e is selected such that

$$\gcd(e, \phi(n)) = 1. \quad (3)$$

Finally, the private exponent d is computed as the modular inverse of e :

$$d \equiv e^{-1} \pmod{\phi(n)}. \quad (4)$$

1.1.3 Implementation

The method `generateRSAKeyPair()` performs the following steps:

1. Select two distinct prime numbers from the predefined prime list.
2. Compute $n = pq$.
3. Compute $\phi(n)$.
4. Select the public exponent e .
5. Verify that e and $\phi(n)$ are relatively prime.
6. Compute d using the modular inverse algorithm.
7. Return the generated key pair.

1.1.4 Role in the Project

RSA keys are used for digital signatures and certificate generation.

1.2 Euclidean Algorithm: gcd(long a, long b)

1.2.1 Purpose

The Greatest Common Divisor (GCD) algorithm is used to verify that the public exponent e is relatively prime to $\phi(n)$.

1.2.2 Mathematical Foundation

The Euclidean Algorithm is based on the identity

$$\gcd(a, b) = \gcd(b, a \bmod b). \quad (5)$$

The procedure continues until the second argument becomes zero.

1.2.3 Implementation

The method repeatedly computes the remainder

$$r = a \bmod b, \quad (6)$$

and updates the variables until

$$b = 0. \quad (7)$$

The final value of a is returned as the greatest common divisor.

1.3 Fast Modular Exponentiation: modPower(long base, long exp, long mod)

1.3.1 Purpose

RSA signatures and Diffie-Hellman key exchange require efficient computation of modular powers of the form

$$a^b \bmod n. \quad (8)$$

1.3.2 Mathematical Foundation

The algorithm uses the square-and-multiply technique.

Instead of performing b multiplications, the exponent is processed bit by bit. Repeated squaring is used to generate higher powers efficiently.

For example,

$$a^{13} = a^8 \times a^4 \times a^1 \quad (9)$$

since

$$13 = (1101)_2. \quad (10)$$

1.3.3 Implementation

The algorithm performs the following operations:

1. Initialize the result to 1.
2. While the exponent is non-zero:
 - (a) If the least significant bit is 1, multiply the result by the current base.
 - (b) Square the base.
 - (c) Shift the exponent one bit to the right.
3. Return the final result.

1.4 Modular Inverse Algorithm: modInverse(long a, long m)

1.4.1 Purpose

The modular inverse algorithm computes the RSA private exponent d .

1.4.2 Mathematical Foundation

The objective is to find d satisfying

$$ed \equiv 1 \pmod{\phi(n)}. \quad (11)$$

The Extended Euclidean Algorithm computes integers x and y such that

$$ax + my = \gcd(a, m). \quad (12)$$

Since

$$\gcd(e, \phi(n)) = 1, \quad (13)$$

we obtain

$$ex + \phi(n)y = 1. \quad (14)$$

Therefore,

$$x = e^{-1} \pmod{\phi(n)}, \quad (15)$$

which becomes the RSA private exponent.

1.4.3 Implementation

The method iteratively updates coefficients until the modular inverse is obtained. If the result is negative, it is converted to the equivalent positive value.

1.5 XOR-Based Hash Function: simpleHash(String data)

1.5.1 Purpose

The hash function generates a compact representation of a message before it is digitally signed.

1.5.2 Mathematical Foundation

Given message bytes

$$b_1, b_2, \dots, b_n \quad (16)$$

the hash value is computed as

$$H = b_1 \oplus b_2 \oplus \dots \oplus b_n \quad (17)$$

where $\oplus$ denotes the XOR operation.

1.5.3 Implementation

The input string is converted into bytes. Each byte is then XORed with the current hash value until a single 8-bit result remains.

1.6 RSA Digital Signature Generation: sign(String data, RSAKeyPair keys)

1.6.1 Purpose

Digital signatures provide authentication, integrity, and non-repudiation.

1.6.2 Mathematical Foundation

First, the message hash is computed:

$$h = H(m). \quad (18)$$

The signature is then generated using the RSA private key:

$$S = h^d \bmod n. \quad (19)$$

1.6.3 Implementation

The method performs two operations:

1. Compute the message hash.
2. Apply modular exponentiation using the RSA private exponent.

1.6.4 Role in the Project

Digital signatures are used by the Certificate Authority (CA) and by users during the authenticated Diffie-Hellman protocol.

1.7 RSA Digital Signature Verification:

`verify(String data, long signature, long e, long n)`

1.7.1 Purpose

Signature verification confirms both message integrity and sender authenticity.

1.7.2 Mathematical Foundation

The verifier computes

$$h_1 = H(m), \tag{20}$$

and recovers

$$h_2 = S^e \bmod n. \tag{21}$$

If

$$h_1 = h_2 \tag{22}$$

the signature is considered valid.

1.7.3 Implementation

The verification process consists of:

1. Recomputing the hash of the received data.
2. Recovering the signed hash using the public key.
3. Comparing the two values.

1.7.4 Security Interpretation

A successful verification indicates that:

- the message has not been modified;
- the signature was generated by the holder of the corresponding private key.

2 Certificate Authority and Certificate Management

2.1 Certificate Authority (CA)

2.1.1 Purpose

A Certificate Authority (CA) is a trusted entity responsible for binding a user's identity to a public key. The CA issues digitally signed certificates that allow other users to verify the authenticity of public keys.

2.1.2 Role in the Project

In this project, the CA performs the following tasks:

1. Generates its own RSA key pair.
2. Issues certificates for users.
3. Signs certificate contents using its private key.
4. Verifies issued certificates using its public key.

All participants trust the CA's public key. Consequently, any certificate successfully verified using this public key is considered authentic.

2.2 Certificate Issuance Algorithm: `issueCertificate(User user)`

2.2.1 Purpose

The certificate issuance algorithm creates a digital certificate that binds a user's identity to their RSA public key.

This allows other users to verify that a public key genuinely belongs to the claimed owner.

2.2.2 Certificate Structure

Each certificate contains:

$$Certificate = (ID, n, e, Signature), \quad (23)$$

where:

- ID is the user identifier.
- n is the RSA modulus.
- e is the RSA public exponent.
- $Signature$ is the CA's digital signature.

2.2.3 Mathematical Foundation

The CA first constructs the certificate data:

$$String = ID \parallel n \parallel e, \quad (24)$$

where $\parallel$ denotes concatenation.

The hash of the certificate data (string) is then computed:

$$h = H(String). \quad (25)$$

The CA signs the hash using its RSA private key:

$$Signature = h^{d_{CA}} \bmod n_{CA} \quad (26)$$

The resulting signature is embedded within the certificate.

2.2.4 Implementation

The method `issueCertificate()` performs the following steps:

1. Retrieve the user's identifier.
2. Retrieve the user's RSA public key components (n, e) .
3. Concatenate the fields into a single string.
4. Generate a digital signature using the CA's private key.
5. Create and return a certificate object.

2.2.5 Security Contribution

The certificate ensures that the public key cannot be replaced without invalidating the CA's signature. Any modification of the certificate contents causes signature verification to fail.

2.3 Certificate Verification Algorithm: `verifyCertificate(Certificate certificate)`

2.3.1 Purpose

Certificate verification determines whether a certificate was genuinely issued by the trusted Certificate Authority and whether its contents have been altered.

2.3.2 Mathematical Foundation

The verifier reconstructs the certificate data:

$$String = ID \parallel n \parallel e, \quad (27)$$

and computes:

$$h_1 = H(String). \quad (28)$$

The signed hash is then recovered using the CA's public key:

$$h_2 = Signature^{e_{CA}} \bmod n_{CA}. \quad (29)$$

If

$$h_1 = h_2 \quad (30)$$

the certificate is considered valid.

2.3.3 Implementation

The method `verifyCertificate()` performs the following operations:

1. Extract the certificate fields.
2. Reconstruct the original certificate data.
3. Compute the expected hash.
4. Recover the signed hash using the CA public key.
5. Compare the two values.

A certificate is accepted only if both hashes are identical.

2.3.4 Security Interpretation

A successful verification guarantees that:

- the certificate was issued by the trusted CA;
- the certificate contents have not been modified;
- the contained public key belongs to the claimed user.

2.4 Trust Model of the PKI

The security of the entire Public Key Infrastructure depends on the trust placed in the Certificate Authority.

All users trust the CA's public key and use it to verify certificates. Therefore, trust in a user's public key is derived from trust in the CA.

This trust relationship can be summarized as:

$$User \rightarrow Certificate \rightarrow CA \quad (31)$$

If the CA's signature is valid, the user's public key is accepted as authentic.

3 Authenticated Diffie-Hellman Key Exchange

3.1 Purpose

The objective of the Diffie–Hellman protocol is to allow two parties to establish a common secret key over an insecure communication channel.

A fundamental limitation of the basic Diffie–Hellman protocol is its vulnerability to Man-in-the-Middle (MITM) attacks. An attacker can intercept and replace public values exchanged between the two parties, thereby creating independent keys with each victim.

To address this weakness, the implementation combines Diffie–Hellman with digital certificates and RSA signatures. As a result, both confidentiality and authentication are achieved.

3.2 Public Parameters

The protocol uses two public constants:

$$p = 65521 \tag{32}$$

$$g = 11, \tag{33}$$

where:

- p is a large prime modulus,
- g is a generator of the multiplicative group modulo p .

These values are publicly known to all participants.

3.3 Certificate Validation Phase

3.3.1 Purpose

Before any key exchange takes place, each participant must verify the authenticity of the other participant's certificate.

This step ensures that the public keys used during authentication genuinely belong to the claimed users.

3.3.2 Implementation

The protocol begins by verifying both certificates:

```
verifyCertificate(A)  
verifyCertificate(B).
```

If either certificate fails verification, the protocol terminates immediately.

3.3.3 Security Contribution

Certificate validation prevents attackers from introducing fraudulent public keys into the protocol.

Without this step, authentication would be meaningless because participants could not trust the public keys they receive.

3.4 Private Key Generation

3.4.1 Purpose

Each participant generates a secret Diffie-Hellman exponent.

These values remain private and are never transmitted through the network.

3.4.2 Mathematical Foundation

Alice selects:

$$a \in [2, p - 1]. \quad (34)$$

Bob selects:

$$b \in [2, p - 1]. \quad (35)$$

Both values are generated randomly.

3.4.3 Implementation

The implementation generates:

```
a = 2 + random.nextInt(65519)
b = 2 + random.nextInt(65519),
```

which ensures that both values lie in the interval

$$2 \leq a, b \leq 65520 \quad (36)$$

3.5 Diffie-Hellman Public Value Generation

3.5.1 Purpose

Each participant computes a public value derived from the private exponent.

These values can be transmitted openly without revealing the secret exponents.

3.5.2 Mathematical Foundation

Alice computes

$$A = g^a \bmod p. \quad (37)$$

Bob computes

$$B = g^b \bmod p. \quad (38)$$

3.5.3 Implementation

The implementation uses the fast modular exponentiation algorithm:

```
A_pub = modPower(G,a,P)
B_pub = modPower(G,b,P).
```

3.6 Public Value Authentication

3.6.1 Purpose

The public Diffie-Hellman values must be authenticated before they are accepted.

Otherwise, an attacker could replace these values and launch a Man-in-the-Middle attack.

3.6.2 Mathematical Foundation

Alice signs her public value:

$$Sig_A = H(A)^{d_A} \bmod n_A. \quad (39)$$

Bob signs his public value:

$$Sig_B = H(B)^{d_B} \bmod n_B, \quad (40)$$

where:

- $H(\cdot)$ is the hash function,
- (d_A, n_A) is Alice's private RSA key,
- (d_B, n_B) is Bob's private RSA key.

3.6.3 Implementation

The signatures are generated as follows:

```
sigA = sign(A_pub, AlicePrivateKey)
sigB = sign(B_pub, BobPrivateKey).
```

3.7 Signature Verification

3.7.1 Purpose

Each participant verifies that the received Diffie-Hellman value was produced by the legitimate owner of the corresponding certificate.

3.7.2 Mathematical Foundation

Alice's signature is verified using Alice's certified public key:

$$H(A) \stackrel{?}{=} Sig_A^{e_A} \bmod n_A \quad (41)$$

Similarly, Bob's signature is verified using Bob's certified public key:

$$H(B) \stackrel{?}{=} Sig_B^{e_B} \bmod n_B \quad (42)$$

If either verification fails, the protocol terminates.

3.7.3 Implementation

The implementation retrieves the public key from the verified certificate and uses it to validate the signature.

Only authenticated public values are accepted.

3.8 Shared Secret Computation

3.8.1 Purpose

After successful authentication, both participants compute a common secret value.

3.8.2 Mathematical Foundation

Alice computes:

$$K_A = B^a \bmod p. \quad (43)$$

Bob computes:

$$K_B = A^b \bmod p. \quad (44)$$

Substituting the definitions of A and B :

$$K_A = (g^b)^a \bmod p = g^{ab} \bmod p, \quad (45)$$

and

$$K_B = (g^a)^b \bmod p = g^{ab} \bmod p. \quad (46)$$

Therefore,

$$K_A = K_B, \quad (47)$$

and both parties obtain the same shared secret.

3.9 Protocol Correctness

The correctness of the protocol follows directly from the commutative property of exponent multiplication:

$$(g^a)^b = (g^b)^a = g^{ab}. \quad (48)$$

Consequently, both participants derive an identical secret key despite never transmitting the key itself through the communication channel.

3.10 Security Analysis

3.10.1 Confidentiality

The shared secret is never transmitted over the network.

Only the public values

$$A = g^a \bmod p \quad (49)$$

and

$$B = g^b \bmod p \quad (50)$$

are exchanged.

An attacker observing the communication cannot efficiently compute the secret key without solving the Discrete Logarithm Problem.

3.10.2 Authentication

Authentication is achieved through RSA digital signatures.

Since only the legitimate user possesses the corresponding private RSA key, successful signature verification proves ownership of the Diffie-Hellman public value.

3.10.3 Integrity

Any modification of a signed Diffie-Hellman public value changes its hash and causes signature verification to fail.

Therefore, tampering with exchanged messages is detected immediately.

3.10.4 Resistance to Man-in-the-Middle Attacks

The traditional Diffie-Hellman protocol does not authenticate exchanged public values and is therefore vulnerable to Man-in-the-Middle attacks.

In the implemented protocol:

1. Certificates are verified using the trusted CA.
2. Diffie-Hellman public values are digitally signed.
3. Signatures are verified using certified public keys.

As a result, an attacker cannot replace public values without possessing the corresponding RSA private keys.

Consequently, the Man-in-the-Middle attack is prevented.

3.11 Role in the Project

This protocol represents the final stage of the system. It combines the Certificate Authority, RSA digital signatures, certificate verification, and Diffie-Hellman key exchange into a complete authenticated key establishment mechanism.

The resulting shared secret can subsequently be used as a symmetric encryption key for secure communication.

4 Answers to the Questions on the Hash Function

4.1 Why Is This Scheme Vulnerable to Signature Forgery Attacks?

The implemented digital signature scheme signs only the hash value of a message rather than the message itself. Therefore, the security of the signature scheme depends directly on the security of the underlying hash function.

The hash function used in this project is a simple XOR-based function:

$$H = b_1 \oplus b_2 \oplus \cdots \oplus b_n. \quad (51)$$

This function produces only an 8-bit output, which means that it can generate at most

$$2^8 = 256. \quad (52)$$

different hash values.

Because the output space is extremely small, finding two different messages with the same hash value is very easy.

Suppose an attacker finds two messages m and m' such that

$$m \neq m' \quad (53)$$

but

$$H(m) = H(m'). \quad (54)$$

If a legitimate user signs message m , the generated signature will be

$$S = H(m)^d \bmod n. \quad (55)$$

Since

$$H(m) = H(m'), \quad (56)$$

the same signature will also verify successfully for message m' .

Consequently, an attacker can replace the original message with another message having the same hash value and reuse the legitimate signature.

This attack is known as a collision-based signature forgery attack.

Therefore, the implemented scheme is vulnerable because the hash function is not collision resistant.

4.2 Which Hash Functions Are Used in Real-World Systems and Why?

Real-world cryptographic systems use secure cryptographic hash functions such as

- SHA-256
- SHA-384
- SHA-512
- SHA-3

instead of simple XOR-based constructions.

These hash functions are preferred because they satisfy the fundamental security requirements of cryptographic hashing:

1. Preimage Resistance

Given a hash value h , it should be computationally infeasible to find a message m such that

$$H(m) = h. \quad (57)$$

2. Second-Preimage Resistance

Given a message m , it should be computationally infeasible to find another message m' such that

$$H(m) = H(m'). \quad (58)$$

3. Collision Resistance

It should be computationally infeasible to find any two distinct messages that produce the same hash value.

For example, SHA-256 produces a 256-bit output and currently has no practical collision attacks. As a result, it is widely used in TLS, digital signatures, certificates, cryptocurrencies, and many other security applications.

Therefore, modern systems rely on SHA-2 and SHA-3 family hash functions because they provide strong security guarantees and resistance against known attacks.

4.3 What Is the Minimum Hash Output Length Required for 80-Bit Security?

To answer this question, it is necessary to consider the Birthday Attack.

For a hash function with an output length of n bits, collision attacks can be performed in approximately

$$2^{n/2} \quad (59)$$

operations due to the Birthday Paradox.

If the desired collision resistance is 80 bits, then the attack complexity should be at least

$$2^{80}. \quad (60)$$

Therefore,

$$2^{n/2} = 2^{80} \quad (61)$$

which implies

$$n = 160. \quad (62)$$

Hence, the minimum hash output length required to achieve approximately 80-bit collision security is

$$\boxed{160 \text{ bits}} \tag{63}$$

This is the reason why older hash functions such as SHA-1 produced a 160-bit output.

However, modern standards typically require much stronger security levels, and hash functions with 256-bit outputs (such as SHA-256) are now preferred.

5 Sequence Diagrams of the Protocol

5.1 Certificate Authority: Certificate Issuance

Figure 1: Diagram of the Certificate Authority (CA) issuing and validating a digital certificate for a user.

Explanation This diagram illustrates the process of certificate issuance and verification in the Public Key Infrastructure (PKI) model.

First, the user submits their identity and RSA public key components (n, e) to the Certificate Authority (CA). The CA constructs a certificate by concatenating the user's identity with the public key values and computing a cryptographic hash over this data.

The CA then signs the hash using its private RSA key, producing a digital signature that binds the user's identity to their public key. The final certificate consists of the tuple $(ID, n, e, \text{Signature})$.

Any party can later verify the authenticity of the certificate using the CA's public key. If the verification succeeds, the public key is considered trustworthy.

5.2 Authenticated Diffie-Hellman Key Exchange

Figure 2: Diagram of the authenticated Diffie-Hellman key exchange using RSA-based digital signatures and certificate validation.

Explanation This diagram shows the authenticated Diffie-Hellman key exchange protocol, which combines symmetric key agreement with public-key authentication.

Before initiating the key exchange, both parties verify each other's certificates using the trusted Certificate Authority (CA). This ensures that the public keys used in the protocol are authentic.

After successful verification, Alice and Bob independently generate private random exponents a and b . They compute their public Diffie-Hellman values as $A = g^a \bmod p$ and $B = g^b \bmod p$.

To prevent Man-in-the-Middle attacks, both values are digitally signed using RSA signatures. The signed values are exchanged and verified using the certified public keys.

Finally, both parties compute the shared secret:

$$K = g^{ab} \bmod p.$$

Since both computations yield the same result, a secure shared key is established.

6 Experimental Results

6.1 End-to-End Execution of the Authenticated Diffie–Hellman Protocol

To evaluate the correctness and security of the implemented system, a complete execution scenario involving a Certificate Authority (CA), Alice, and Bob was performed.

The following output demonstrates the step-by-step execution of the protocol:

```
=== Diffie-Hellman Authenticated Protocol Start ===

Verifying certificate of A...
A certificate VALID

Verifying certificate of B...
B certificate VALID

Private values generated:
a (Alice secret) generated
b (Bob secret) generated

Public values computed:
A_pub =  $g^a \bmod p$  = 49799
B_pub =  $g^b \bmod p$  = 20312

Digital signatures created:
Alice signs A_pub
Bob signs B_pub

Verifying Alice signature...
Alice signature VALID

Verifying Bob signature...
Bob signature VALID

Computing shared secret...

Shared key computed successfully:
K_A = 44619
K_B = 44619

*** PROTOCOL SUCCESSFUL ***
Secure shared secret established.
K = 44619
```

6.2 Result Analysis

The experimental results confirm the correctness of the implemented authenticated Diffie-Hellman protocol.

Both participants independently compute the same shared secret value:

$$K_A = K_B = g^{ab} \bmod p \quad (64)$$

This demonstrates that the Diffie-Hellman key exchange works correctly.

Furthermore, all certificates are successfully verified by the Certificate Authority, and all RSA signatures are validated before key computation.

This ensures that:

- The communicating parties are authenticated.
- The exchanged Diffie-Hellman values are not tampered with.
- The system is resistant to Man-in-the-Middle attacks under the assumed model.

Therefore, the implementation successfully achieves both confidentiality and authentication.

7 Feistel-Based Symmetric Encryption System

7.1 Purpose

After the authenticated Diffie-Hellman protocol establishes a shared secret key, a symmetric encryption algorithm is required to provide message confidentiality.

The project implements a custom Feistel-based block cipher that uses the shared master key generated during the authenticated Diffie-Hellman phase. The cipher encrypts plaintext messages before transmission and decrypts them at the receiver side.

The implementation follows all requirements specified in the project description, including 16-bit blocks, four Feistel rounds, round-key generation, S-Box substitution, bit permutation, and ISO/IEC 7816-4 padding.

7.2 Feistel Network Structure

7.2.1 Block Configuration

The cipher operates on 16-bit blocks:

[BlockSize = 16 bits]

Each block is divided into:

- Left Half (L): 8 bits
- Right Half (R): 8 bits

The encryption process consists of four Feistel rounds.

7.2.2 Mathematical Foundation

For round i :

$$L_{i+1} = R_i$$

$$R_{i+1} = L_i \oplus F(R_i, K_i)$$

where:

- L_i and R_i represent the left and right halves,
- K_i denotes the round key,
- F denotes the round function.

After the final round, the resulting halves are concatenated to produce the ciphertext block.

7.2.3 Correctness

One of the main advantages of the Feistel structure is that decryption uses the same algorithm as encryption, with the only difference being that round keys are applied in reverse order.

Therefore, the original plaintext can always be recovered correctly.

7.3 Round Function F

7.3.1 Purpose

The round function introduces confusion and diffusion into the cipher.

7.3.2 XOR with Round Key

Given the right half R and round key K :

$$X = R \oplus K$$

This operation incorporates the secret key into the encryption process.

7.3.3 S-Box Substitution

The resulting byte is separated into two 4-bit halves:

$$HighNibble = X \gg 4$$

$$LowNibble = X \& 0x0F$$

Each nibble is transformed using the project S-Box shown in Table 1.

The S-Box introduces non-linearity and increases resistance against simple cryptanalytic attacks.

Table 1: S-Box Used in the Feistel Cipher

Input	Output
0	12
1	5
2	6
3	11
4	9
5	0
6	10
7	13
8	3
9	14
10	15
11	8
12	4
13	7
14	1
15	2

7.3.4 Bit Permutation

After substitution, the bits are permuted according to:

$$P = [0, 4, 2, 6, 1, 5, 3, 7]$$

The permutation spreads local changes throughout the output and improves diffusion.

7.3.5 Security Contribution

The combination of:

1. XOR key mixing
2. Non-linear substitution
3. Bit permutation

provides the essential cryptographic properties required for block cipher design.

7.4 Round Key Generation

7.4.1 Purpose

Each Feistel round requires a distinct round key derived from the shared master key.

7.4.2 Mathematical Foundation

Let K denote the 16-bit master key obtained from the authenticated Diffie–Hellman protocol.
Round key i is generated as:

$$K_i = ((K \gg i) \oplus (K \gg (i + 4))) \& 0xFF$$

for:

$$i \in \{1, 2, 3, 4\}$$

7.4.3 Implementation

The implementation derives four independent 8-bit round keys.

Using different round keys ensures that each Feistel round performs a different transformation.

7.5 Message Encoding

7.5.1 ASCII Conversion

User messages are initially represented as character strings.

Each character is converted to its ASCII value and then represented as an 8-bit binary sequence.

Example:

['H' = 72 = 01001000]

['i' = 105 = 01101001]

The resulting binary sequences are concatenated into a single bit stream.

7.5.2 Block Formation

The bit stream is divided into consecutive 16-bit blocks before encryption.

Each block is processed independently by the Feistel cipher.

7.6 ISO/IEC 7816-4 Padding

7.6.1 Purpose

Since the message length may not be an exact multiple of the block size, padding is required.

7.6.2 Padding Procedure

The implementation follows ISO/IEC 7816-4.

The procedure is:

1. Append byte 10000000.
2. Append zero bytes until the total length becomes a multiple of 16 bits.

Padding is always added, even when the original message length already aligns perfectly with the block boundary.

7.6.3 Example

Message:

Hi

Binary representation:

[01001000 01101001]

Padding:

[10000000 00000000]

Final blocks:

[0100100001101001]

[1000000000000000]

7.6.4 Padding Removal

During decryption:

1. The binary stream is reconstructed.
2. Trailing zero bytes are removed.
3. The first encountered byte equal to 10000000 is removed.
4. The original plaintext message is recovered.

7.7 Security Analysis

The implemented cipher is intended for educational purposes and demonstrates the principles of modern block cipher design.

The cipher provides:

- Key-dependent transformations
- Multiple encryption rounds
- Non-linear substitution
- Bit-level diffusion

However, it should not be considered secure for real-world deployment because:

- The block size is only 16 bits.
- The key size is only 16 bits.
- The number of rounds is limited.
- Modern block ciphers employ significantly stronger designs.

Despite these limitations, the implementation successfully demonstrates the core concepts of symmetric cryptography.

7.8 Experimental Results

7.8.1 Encryption Test

After the authenticated Diffie–Hellman protocol generated the shared master key, a plaintext message was encrypted using the Feistel cipher.

Example message:

Hello TA

The message was converted into binary form, padded using ISO/IEC 7816-4, divided into 16-bit blocks, and encrypted successfully.

The program produced a sequence of encrypted integer blocks.

7.8.2 Decryption Test

The encrypted blocks were supplied to the decryption module.

After applying Feistel decryption, removing padding, and converting the recovered binary stream back into ASCII characters, the original plaintext message was reconstructed successfully.

7.8.3 Result Analysis

The experimental results confirm that:

- Binary encoding functions correctly.
- ISO/IEC 7816-4 padding is applied and removed correctly.
- Feistel encryption and decryption operate correctly.
- The original plaintext is recovered without loss of information.
- The shared key generated during the authenticated Diffie–Hellman protocol is successfully used as the symmetric encryption key.

Therefore, the implementation successfully fulfills all Phase 3 requirements of the project and completes the secure messaging system.

8 User Interface and System Integration

8.1 Purpose

The final phase of the project focuses on integrating all cryptographic components into a single interactive application.

The objective of the user interface is to provide a simple mechanism for executing the complete secure communication workflow while hiding implementation details from the user.

The interface connects the following components:

- RSA Key Generation
- Certificate Authority (CA)
- Certificate Verification

- Authenticated Diffie–Hellman Key Exchange
- Feistel Encryption
- Feistel Decryption

All components are executed within a single Java application.

8.2 System Architecture

The implementation follows a modular architecture consisting of multiple packages:

- `phase1` – RSA, PKI, and Certificate Management
- `phase2` – Authenticated Diffie–Hellman
- `phase3` – Feistel Cipher and Padding
- `app` – Integration and Application State Management
- `ui` – Console-Based User Interface

This design separates cryptographic functionality from user interaction and improves maintainability.

8.3 Application State Management

A dedicated application state object stores the current system status.

The stored information includes:

- CA Public Key
- User A Public Key
- User B Public Key
- User Certificates
- Certificate Verification Status
- Shared Master Key
- Last Encrypted Message Blocks

This approach allows different menu operations to access previously generated cryptographic data.

8.4 Menu Structure

The application provides a console-based menu consisting of six options.

1. Initialize System
2. Establish Shared Key
3. Encrypt Message
4. Decrypt Message
5. Show Current Status
6. Exit

The menu remains active until the user chooses the exit option.

8.5 Initialize System

The first menu option performs the initialization phase.

The following operations are executed:

1. Generate the CA RSA key pair.
2. Generate User A RSA key pair.
3. Generate User B RSA key pair.
4. Issue certificates for both users.
5. Verify generated certificates.
6. Store all public information in the application state.

Successful completion of this phase indicates that the PKI infrastructure has been established.

8.6 Establish Shared Key

The second menu option executes the authenticated Diffie–Hellman protocol.

The following operations are performed:

1. Validate both certificates.
2. Generate Diffie–Hellman private values.
3. Compute public Diffie–Hellman values.
4. Generate RSA signatures.
5. Verify signatures.
6. Compute the shared secret.
7. Derive the 16-bit master key.

The generated master key becomes the symmetric key used by the Feistel cipher.

8.7 Message Encryption

The third menu option encrypts a plaintext message.

The workflow is:

1. Receive plaintext input from the user.
2. Convert characters to ASCII binary representation.
3. Apply ISO/IEC 7816-4 padding.
4. Divide the message into 16-bit blocks.
5. Encrypt each block using the Feistel network.
6. Display encrypted blocks.
7. Store encrypted blocks for future decryption.

8.8 Message Decryption

The fourth menu option decrypts encrypted message blocks.

The workflow is:

1. Receive encrypted blocks.
2. Apply Feistel decryption.
3. Reconstruct the binary stream.
4. Remove ISO/IEC 7816-4 padding.
5. Convert ASCII values back to characters.
6. Display the recovered plaintext.

Successful execution demonstrates the correctness of the implemented symmetric cipher.

8.9 Status Monitoring

The fifth menu option displays the current system status.

The following information is shown:

- CA Public Key
- User A Public Key
- User B Public Key
- Certificate Verification Status
- Current Master Key
- Last Encrypted Ciphertext

This functionality assists in debugging and validating the protocol execution.

8.10 Experimental Demonstration

A complete end-to-end execution was performed using the integrated user interface.

The following sequence was executed:

1. Initialize System
2. Establish Shared Key
3. Encrypt Message
4. Decrypt Message
5. Display Current Status

The execution confirmed that:

- Certificates are generated and verified successfully.
- The authenticated Diffie–Hellman protocol establishes a valid shared secret.
- The Feistel cipher encrypts plaintext correctly.
- The decryption process recovers the original message.
- All components operate correctly within a single integrated application.

8.11 Conclusion

The final integrated application successfully combines public-key cryptography, certificate management, authenticated key exchange, and symmetric encryption into a single secure messaging system.

The user interface provides a convenient mechanism for executing the entire protocol while maintaining a modular software architecture.

Therefore, the project satisfies all functional requirements described in the assignment specification.